

Git для пацанов

Практическое руководство по Git и GitHub для тех, кто хочет разобраться без воды и скуки

[\[Иллюстрация: License: WTFPL\]](#) [\[Иллюстрация: Язык\]](#) [\[Иллюстрация: Уровень\]](#) [\[Иллюстрация: Платформа\]](#) [\[Иллюстрация: Git\]](#)

Об авторе

Книжка написана **живым человеком** с помощью ИИ как соавтора и инструмента.

Автор: [@mamkincoderr](#)

"Не учебник для академиков. Объясняю так, как объяснил бы другу — прямо, с юмором, без занудства."

О чём эта книжка

Ты открываешь терминал, вводишь `git push` — и что-то идёт не так. Тимлид говорит "ребейзни на мейн" — ты киваешь, но не понимаешь. Знакомо?

Эта книга — для тех, кто хочет перестать бояться Git и начать работать как нормальный разработчик. Без воды, с реальными командами, с объяснениями на русском и с шутками, которые понятны нашим.

Что ты получишь: - Понимание всего Git-жаргона (больше никакого "чё?") - Рабочий Git на своём компе с нуля - Уверенные базовые команды каждого дня - Понимание веток и зачем они нужны - Умение работать в команде через GitHub и Pull Request-ы

Содержание

#	Урок	Что внутри
1	Жаргон — говоришь как свой	Репа, коммит, пуш, ПР, форк, апстрим, конвеншенал коммитс
2	Ставим Git и делаем первый коммит	Установка Git на Windows, настройка, init, add, commit, log
3	Подключаем GitHub	Аккаунт, PAT и SSH аутентификация, remote add, первый push
4	Команды на каждый день	status, add, commit, push, log, stash, .gitignore, откат изменений
5	Ветки — работаешь без страха	Создание веток, merge vs rebase, конфликты, naming conventions
6	GitHub и командная работа	Pull Requests, форки, Issues, Actions
7	Git в VSCode	Source Control панель, stage/commit/diff мышкой, конфликты

Среда и требования

Все команды написаны и проверены на: - **Windows 11** - **Git 2.50.1** (рекомендуется 2.23+ — используем `git switch` вместо старого `git checkout`) - **PowerShell** (не cmd)

Если у тебя Linux или macOS — большинство команд те же. Там где есть отличия, в тексте есть раскрывающиеся блоки **Linux / macOS** с альтернативными командами.

Для кого

- Начинаешь программировать и хочешь наконец-то понять Git
- Уже что-то умеешь, но Git всё ещё пугает
- Знаешь команды, но не понимаешь что за ними стоит
- Хочешь работать в команде и не облажаться

Не нужно знать: ничего. Серьёзно, с нуля.

Как читать

Уроки идут по порядку — каждый следующий опирается на предыдущий.

1. Начни с [Урока 1](#) — даже если кажется что знаешь жаргон
 2. На каждом уроке есть **практические задания** — делай их, не пропускай
 3. Завёл `git-practice` папку? Отлично, именно в ней и практикуешься
-

Хочешь помочь

Нашёл ошибку? Знаешь как объяснить лучше? Открывай [Issue](#) или Pull Request — всё приветствуется.

Лицензия

[WTFPL](#) — делай что хочешь.

Сделано с  и помощью ИИ · [@mamkincoderr](#)

Урок 1: Git-жаргон для пацанов из Рунета

TL;DR: Учишь слова — потом не будешь тупить когда тимлид скажет "ребейзни на мейн и открой ПР"

Йо, братан! Перед тем как нырять в команды, давай разберёмся, на каком языке вообще говорят эти гит-хабовские чуваки. В Рунете это всё звучит особенно эпично.

Основной сленг, который ты услышишь в чатах и на митапах

Репозиторий (Repository / Repo)

Твоя папка с кодом, где Git ведёт учёт всех изменений. "Залей на репу", "Форкни репу".

Думай об этом как о папке, которая помнит всё. Прямо всё. Даже тот стыдный код в 3 ночи.

Коммит (Commit)

Снимок проекта. Ты "закоммитил" изменения — сохранил момент.

Runet-вариант: "Закоммить", "коммитнул", "сделай коммит".

Это как *quicksave* в игре. Умрёшь — вернёшься сюда.

Пуш (Push)

Отправил свои коммиты на GitHub. "Запушь ветку", "пушь в main".

Пока не запушил — это существует только у тебя на ноуте. Упадёт жёсткий диск — всё, пока код.

Пулл (Pull)

Скачал изменения с сервера. "Сделай пулл перед работой, чтобы не было конфликтов".

Не сделал пулл с утра — огребёшь конфликтов к обеду. Проверено миллионами разработчиков.

Мерж (Merge)

Слияние веток. "Замержи в main".

Два потока кода встречаются. Иногда дружно, иногда — война на 300 строк.

Ребейз (Rebase)

Более крутой способ слить — "переставляешь" свои коммиты поверх свежей базы. "Лучше ребейз сделай, чтобы история была чистой".

Мерж — это когда два поезда сцепились. Ребейз — когда ты пересел в другой поезд как будто всегда там и ехал.

Бранч / Ветка (Branch)

Параллельная линия разработки. "Свитчнись на бранч", "создай фичу-бранч".

Ты пишешь фичу в своём мире, не ломая продакшн. Красота.

ПР (Pull Request)

Запрос на вливание твоего кода. Главный инструмент код-ревью. "Открой ПР", "замержь ПР".

Ты говоришь команде: "Эй, посмотрите что я накодил". Они смотрят. Иногда плачут.

Форк (Fork)

Сделал свою копию чужого проекта. "Форкни репозиторий и отправь ПР".

Это как взять чужой рецепт, сделать по-своему и предложить автору попробовать твою версию.

Апстрим (Upstream)

Оригинальная репа, от которой ты форкнул. "Добавь upstream".

Твоя копия — это ты. Апстрим — это тот с кого ты скопировал. Не забывай синкаться.

Конфликт (Merge Conflict)

Когда два человека изменили один и тот же кусок кода. Git говорит: "Разберись, чел".

Самый неприятный момент в жизни разработчика после "works on my machine".

HEAD

Указатель, где ты сейчас находишься. "HEAD detached" — ты в отрыве от ветки.

"HEAD detached" звучит страшно, но это просто значит ты смотришь на старый коммит. Не паникуй.

Сташ (Stash)

Спрятал незаконченные изменения. "Сташни, потом поп".

Рюкзак для незаконченного кода. Кинул туда — потом достал. Удобно когда тебя просят срочно пофиксить баг, а у тебя полурабочая фича.

Ориджин (Origin)

Стандартное имя твоего удалённого репозитория на GitHub.

Просто псевдоним для длинной ссылки на GitHub. Чтобы не печатать `https://github.com/...` каждый раз.

Cherry-pick

Взял один конкретный коммит из другой ветки. "Cherry-pick вот тот фикс".

Как вишенку с торта — берёшь только то, что хочешь, без всего остального.

Конвеншенал коммитс (Conventional Commits)

Стандарт написания сообщений коммитов. Выглядит так:

```
тип(область): что сделал

feat(auth): добавил авторизацию через VK
fix(cart): исправил падение при пустой корзине
docs: обновил README
refactor(api): вынес логику в отдельный сервис
chore: обновил зависимости
test(login): добавил тесты формы входа
perf(images): включил ленивую загрузку
```

На первый взгляд — просто красивые префиксы. На самом деле это **машиночитаемый формат**.

Инструменты вроде [standard-version](#) или [semantic-release](#) умеют:

- **Авто-генерировать CHANGELOG.md** — из коммитов за версию, без ручной писанины
- **Авто-выбирать номер версии** по [SemVer](#): `feat:` → minor (`1.0` → `1.1`), `fix:` → patch (`1.1.0` → `1.1.1`), `feat!:` (breaking change) → major (`1.1` → `2.0`)
- **Автоматически выпускать релиз** — тег + GitHub Release + публикация в npm

Пишешь коммиты по стандарту → CI сам определяет версию, пишет changelog и выкатывает релиз. Ты ничего руками не делаешь.

В Рунете часто игнорируют это пока не приходится вести changelog вручную перед каждым релизом. Один раз настроил — больше не думаешь.

Фразы, которые ты будешь слышать каждый день

- "Сделай ребейз на main, а то история как после взрыва"
- "Пушь форс, но с `--force-with-lease`, не убей мне репу!"
- "Открыл ПР, жду ревью"
- "Конфликты на 300 строк, помогай"

- "Репа в зелёном, можно мержить"
- "Этот код — легаси, давай рефакторить"
- "Не пушь в main напрямую, ты что, стажёр?"
- "Сташни и переключись, мне срочно нужна твоя помощь"

Типичные ошибки новичка

Ошибка	Что происходит
Пушишь прямо в <code>main</code>	Тимлид находит тебя и разговаривает по душам
Коммитишь <code>.env</code> файл с паролями	Мгновенно меняешь все пароли и молишься
Пишешь <code>fix</code> в каждом коммите	Никто не понимает что вообще происходило
Не делаешь <code>pull</code> перед работой	Конфликты на конфликтах
<code>git push --force</code> без <code>--lease</code>	Удаляешь чужую работу. Тебя не любят

Практика прямо сейчас

В своём `git-practice` создай файл `GLOSSARY.md` и запиши туда 8–10 терминов своими словами в прикольном стиле. Объясни как будто рассказываешь другу-нубу за чаем.

Теперь ты говоришь на одном языке с разработчиками!

Переходим к настоящим урокам. Готов рвать Git?

Урок 2: Ставим Git и делаем первый коммит

TL;DR: Ставишь Git, говоришь кто ты, создаёшь папку, пишешь `git init` — и у тебя уже своя репа с машиной времени внутри. GitHub пока не нужен.

• **Среда:** Все команды проверены на **Windows 11 с Git 2.50.1**. Используем **PowerShell** (не `cmd`). Если у тебя старее — обнови Git, некоторые команды появились в версии 2.23+.

Git — что это вообще такое

Представь: ты пишешь курсовую. Делаешь копии — `курсовая_финал.docx`, `курсовая_финал2.docx`, `курсовая_точно_финал.docx`, `курсовая_ааааа.docx`.

Git решает эту проблему раз и навсегда.

Git — это программа на твоём компьютере. Она следит за изменениями в файлах и помнит всю историю. Работает полностью офлайн, без интернета, без аккаунтов.

Что умеет: - Запоминать каждое изменение (коммиты) - Возвращать код к любому предыдущему состоянию - Давать параллельные вселенные для разных задач (ветки) - Не убивать друг друга когда работаешь в команде

GitHub — это отдельный сервис в интернете. Он хранит Git-репозитории в облаке. Как Word и Google Docs: Word работает без интернета, Google Docs — облачный сервис поверх него. Git и GitHub — это не одно и то же. До GitHub дойдём в следующем уроке.

Установка Git на Windows

Шаг 1 — Скачай

Перейди на <https://git-scm.com/download/win> и скачай установщик.

Шаг 2 — Установи

Запусти установщик. Везде жми **Next** с настройками по умолчанию — они нормальные.

Единственное что стоит поменять — редактор по умолчанию. Если юзаешь VSCode, выбери его вместо Vim. Vim — это отдельный квест "как выйти из редактора".

Шаг 3 — Проверь

Открой PowerShell и проверь:

```
git --version
# Должно написать что-то типа: git version 2.50.1.windows.1
# Если написало — ты красавчик. Если "команда не найдена" — переустанови.
```

Linux / macOS — установка Git

****macOS:****

```
# Homebrew (рекомендуется):  
brew install git  
  
# Или встроенные инструменты Apple (чуть старше):  
xcode-select --install
```

****Linux (Ubuntu / Debian):****

```
sudo apt update && sudo apt install git
```

****Linux (Fedora / RHEL):****

```
sudo dnf install git
```

Проверка та же: ``git --version``

Первоначальная настройка

Git должен знать кто ты. Открой **PowerShell** и выполни:

```
# Говоришь Git своё имя — появится в каждом коммите  
git config --global user.name "Твоё Имя"  
  
# Говоришь Git свой email  
git config --global user.email "твой@email.ru"  
  
# Называем главную ветку "main" а не "master" (современный стандарт)  
git config --global init.defaultBranch main  
  
# Фикс переносов строк — важно если в команде разные ОС  
git config --global core.autocrlf true      # Windows  
# git config --global core.autocrlf input  # Linux / macOS  
  
# Открывать VSCode для написания длинных сообщений коммитов  
git config --global core.editor "code --wait"
```

Проверь что всё записалось:

```
git config --global --list  
# Выплюнет список всех настроек
```

***--global** значит "для всех проектов на этом компьютере". Можно делать настройки для конкретного проекта без этого флага — они перекроют глобальные.*

Первый репозиторий — поехали

```
# Создаём папку проекта
mkdir git-practice
cd git-practice

# Инициализируем Git — создаётся скрытая папка .git
git init
# Ответит: "Initialized empty Git repository in ..."
# Всё — репа живёт!

# Смотрим что происходит
git status
# Скажет что нет коммитов и нечего трекать — нормально, только начал
```

Создай первый файл:

```
New-Item README.md
Add-Content README.md "# Мой первый репозиторий"
Add-Content README.md "Здесь будет мой крутой проект."
```

Linux / macOS

```
touch README.md
echo "# Мой первый репозиторий" >> README.md
echo "Здесь будет мой крутой проект." >> README.md
```

```
git status
# Увидишь README.md как "untracked file" — Git видит файл, но ещё не трекает
```

Добавляй в индекс и коммить:

```
# Добавляем файл в индекс (в "зал ожидания" перед коммитом)
git add README.md

git status
# Теперь README.md зелёный — готов к коммиту

# Делаем первый коммит — первый снимок истории
git commit -m "feat: initial commit"
# Git скажет: "[main (root-commit) abc1234] feat: initial commit"
# Первый коммит в репе — поздравляю, пацан!
```

Посмотри историю:

```
git log --oneline
# Увидишь: abc1234 feat: initial commit
# Хеш + сообщение. Это и есть история проекта.
```

Папка `.git` — мозг репы. Не удаляй. Снесёшь — вся история коммитов улетит, останется просто папка с файлами.

Попрактикуйся — закоммить ещё пару раз

Сделай ещё пару изменений чтобы история в репе ожила:

```
# Создаём ещё файл
New-Item index.html
Add-Content index.html "<h1>Привет, мир!</h1>"
```

Linux / macOS

```
touch index.html
echo "<h1>Привет, мир!</h1>" >> index.html
```

```
git add index.html
git commit -m "feat: add index page"

# Меняем README
Add-Content README.md "Стек: HTML, CSS, JS"
git add README.md
git commit -m "docs: update README with stack"

# Смотрим что получилось
git log --oneline
# Три коммита, история живёт
```

Задание к уроку

- Установи Git и проверь `git --version`
- Настрой имя, email, `init.defaultBranch main` и `autocrlf true`
- Создай папку `git-practice` и сделай `git init`
- Создай несколько файлов, добавь их через `git add` и сделай 3 коммита
- Выполни `git log --oneline` и убедись что видишь историю

Домашнее задание

- Создай файл `.gitignore` и добавь туда `*.log` и `.env` — закоммить его
- Попробуй `git log --oneline --graph` — выглядит круто даже с одной веткой
- Позэкспериментируй с `git diff` до и после `git add` — почувствуй разницу

Типичные ошибки новичка

Ошибка	Что происходит
Не настроил имя/email перед коммитом	Коммиты без автора, потом не разберёшься
Удалил папку <code>.git</code>	Репозиторий превратился в обычную папку, вся история улетела
Не настроил <code>autocrlf true</code>	Коллеги на Mac/Linux видят изменения в каждой строке каждого файла
Делаешь <code>git add .</code> не глядя	Добавляешь в коммит всякий мусор
Пишешь <code>fix</code> в каждом коммите	Через месяц не понимаешь что вообще делал

Урок 3: Подключаем удалённый репозиторий

TL;DR: *Git умеет пушить не только на GitHub. Папка в сети, свой сервер, GitLab, Gitea — всё это работает одинаково: `git remote add` + `git push`. GitHub — просто самый известный вариант.*

В прошлом уроке история коммитов жила только на твоём компьютере. Теперь поднимаем её куда-то ещё — чтобы был бэкап, чтобы команда могла подключиться, чтобы код не умер вместе с жёстким диском.

Вариантов куда пушить — несколько. Разберём все.

Вариант 1: Папка в локальной сети — без интернета вообще

Самый простой способ организовать командную работу без интернета, без аккаунтов и без внешних сервисов. Подходит для офиса, домашней сети, закрытого контура.

Идея: создаёшь **bare-репозиторий** (голый репозиторий — без рабочих файлов, только внутренности Git) в общей папке сети. Все пушат туда как в обычный remote.

```
# На машине-сервере или в общей сетевой папке:
git init --bare \\СЕРВЕР\repos\myproject.git
# Или локально, если сетевой диск подключён:
git init --bare Z:\repos\myproject.git
```

Каждый в команде клонирует себе:

```
git clone \\СЕРВЕР\repos\myproject.git
# Или через букву диска:
git clone Z:\repos\myproject.git
```

Linux / macOS — пути к bare-репозиторию

```
# Создать bare-репо на сервере:
git init --bare /srv/git/myproject.git

# Клонировать (если папка примонтирована через NFS или SMB):
git clone /mnt/server/repos/myproject.git

# Клонировать по SSH (самый распространённый способ в Linux):
git clone user@server:/srv/git/myproject.git
```

Дальше — пушишь, пуллишь, бранчишься как обычно. Без лимитов, без аккаунтов, без интернета.

Варе-репозиторий — это просто папка с содержимым `.git` без рабочих файлов. Ты не можешь открыть его и посмотреть код напрямую — это хранилище для Git, не для людей. Именно поэтому его кладут на сервер, а не работают внутри.

Когда это лучший выбор: - Команда в одной сети, интернета нет или он не нужен - Корпоративная среда с запретом внешних сервисов - Хочешь полный контроль без третьих сторон

Вариант 2: Свой Git-сервер с веб-интерфейсом

Если нужен не просто `push/pull`, а нормальный веб-интерфейс с Pull Request-ами, code review, Issues — поднимаешь свой сервер. Это не так страшно как звучит.

Gitea — самый лёгкий вариант

Один бинарный файл, запускается за 5 минут, весит мало, работает на любом VPS или даже на Raspberry Pi.

```
# Скачиваешь бинарник с gitea.io, запускаешь:
.\gitea.exe web
# Открываешь браузер: http://localhost:3000
# Проходишь установщик — готово
```

После настройки — всё как на GitHub: создаёшь репы, пушишь, открываешь ПР. Только адрес твой, не их:

```
git remote add origin http://твой-сервер:3000/USERNAME/myproject.git
git push -u origin main
```

GitLab — для серьёзных команд

Мощнее Gitea, есть встроенный CI/CD, container registry, wiki. Есть облачная версия на gitlab.com и self-hosted. Популярен в корпоративной среде именно потому что не зависит от внешних сервисов.

Когда свой сервер лучший выбор: - Хочешь веб-интерфейс как GitHub, но без GitHub - Корпоративные требования к размещению данных - Нужен полный контроль над доступом и данными

Вариант 3: Облачный сервис — GitHub и не только

Самый простой старт — не надо ничего поднимать, всё уже работает.

Сервис	Для кого
GitHub	Портфолио, open source, большинство команд
GitLab.com	Команды которым нужен встроенный CI/CD
Bitbucket	Если уже используете Jira/Confluence от Atlassian

Сервис	Для кого
Codeberg	Open source, без слежки, Gitea под капотом

Для большинства людей которые читают эту книжку — GitHub. Он самый известный, работодатели смотрят туда, open source живёт там.

Создаём аккаунт на GitHub

Зайди на <https://github.com> и зарегистрируйся — имя, email, пароль.

Выбирай username осторожно — он будет в ссылках на все твои репы и в резюме. **mamkincoderr — норм. **xxx_darkSOUL_2007_xxx** — возможно, не лучший выбор для работодателя.*

Аутентификация на GitHub

Когда делаешь `git push`, GitHub спросит "а ты вообще кто?". Два способа:

PAT (Personal Access Token) — быстро и просто

1. GitHub → **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)**
2. **Generate new token** → выбери срок → поставь галочку **repo**
3. **Скопируй токен сразу** — больше не увидишь его
4. При первом `git push` вставь токен вместо пароля

PAT — временный пропуск в твои репы. Удобно для старта. Храни как пароль — утечёт токен, утечёт доступ ко всем репам.

SSH ключ — один раз и навсегда (рекомендуется)

SSH — криптографическая пара ключей. Приватный остаётся у тебя, публичный отдаёшь GitHub. Больше никаких паролей.

```
# Генерируем ключ (жми Enter на все вопросы) — команда одинакова на всех ОС
ssh-keygen -t ed25519 -C "твой@email.ru"

# Выводим публичный ключ — его копируешь на GitHub
Get-Content "$env:USERPROFILE\.ssh\id_ed25519.pub"
```

Linux / macOS

```
cat ~/.ssh/id_ed25519.pub
```

На GitHub: **Settings** → **SSH and GPG keys** → **New SSH key** → вставляй.

```
# Запускаем ssh-agent и добавляем ключ (один раз)
Set-Service -Name ssh-agent -StartupType Manual
Start-Service ssh-agent
ssh-add "$env:USERPROFILE\.ssh\id_ed25519"

# Проверяем
ssh -T git@github.com

# Должно сказать: "Hi ИМЯ! You've successfully authenticated..."
```

Linux / macOS — запуск ssh-agent

****Linux:****

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

****macOS**** — Keychain сам держит ключи, достаточно добавить один раз:

```
ssh-add --apple-use-keychain ~/.ssh/id_ed25519
```

После перезагрузки ключ подтягивается автоматически — `ssh-agent` запускать вручную не нужно. Проверка везде одинакова:

```
ssh -T git@github.com
```

**SSH — как пропуск на проходной. Один раз оформил, дальше просто проходишь. PAT — как каждый раз предъявлять паспорт.*

Пушим репозиторий на GitHub

Создай репу на GitHub: **New** → дай имя `git-practice` → **Create repository**. Не добавляй README — у тебя уже есть локальная история.

GitHub покажет команды. Нам нужен блок "push an existing repository":

```
# Подключаем GitHub как remote (origin — стандартное имя)
# SSH:
git remote add origin git@github.com:ТВОЙ_USERNAME/git-practice.git

# HTTPS (PAT):
git remote add origin https://github.com/ТВОЙ_USERNAME/git-practice.git

# Пушим (-u запоминает куда пушить, следующий раз просто git push)
git push -u origin main
```

Зайди на GitHub — там уже твоя репа, файлы и вся история коммитов.

```
# Проверить что remote настроен правильно
git remote -v
# origin git@github.com:USERNAME/git-practice.git (fetch)
# origin git@github.com:USERNAME/git-practice.git (push)
```

² **origin** — просто псевдоним для длинной ссылки. Можно назвать как угодно, но **origin** — стандарт которого все придерживаются.

Задание к уроку

1. Создай bare-репозиторий в любой локальной папке и запусти туда свою репу (`git remote add local ./путь/к/bare.git`)
2. Создай аккаунт на GitHub
3. Настрой аутентификацию — PAT или SSH
4. Создай репу на GitHub и подключи через `git remote add origin`
5. Сделай `git push -u origin main` и убедись что коммиты видны на GitHub

Домашнее задание

- Изучи страницу репы на GitHub: вкладки Code, Commits, Settings
- Отредактируй файл прямо на GitHub (кнопка карандаша), сделай коммит через веб — потом `git pull` и посмотри что прилетело
- Попробуй добавить второй remote: `git remote add backup ./local-backup.git` — убедись что `git remote -v` показывает оба

Типичные ошибки новичка

Ошибка	Что происходит
<code>git init --bare</code> в рабочей папке	Создал голый репо там где хотел работать — файлов не видно
Email не совпадает с GitHub	Коммиты не привязываются к профилю
Скопировал HTTPS-ссылку но настроил SSH	<code>git push</code> каждый раз спрашивает пароль
Потерял PAT-токен	Генерируй новый — GitHub не показывает его повторно
Запустил <code>.env</code> в публичную репу	Меняй все секреты срочно — боты сканируют GitHub за минуты

Урок 4: Команды которые ты будешь юзать каждый день

TL;DR: `git add .` → `git commit -m "feat: что сделал"` → `git push` — вот и весь основной цикл. Остальное — нюансы.

Основной цикл работы — запомни как дышать

Каждый день работы с Git выглядит примерно так:

Написал код → Проверил что изменилось → Добавил в индекс → Закоммитил → Запустил

В командах:

```
git status      # Смотришь что вообще происходит
git add .       # Добавляешь ВСЕ изменения в "зал ожидания"
git commit -m "feat: добавил кнопку авторизации" # Делаешь снимок
git push        # Отправляешь на GitHub
```

`git add` — это не сохранение. Это как собрать вещи в чемодан. `git commit` — это когда ты застегнул чемодан и наклеил стикер. `git push` — это когда отправил чемодан на склад (GitHub).

Смотрим что происходит

```
git status
# Показывает:
# - Какие файлы изменены (красные — не в индексе, зелёные — готовы к коммиту)
# - На какой ветке ты сейчас
# - Есть ли непущутые коммиты

git diff
# Что конкретно изменилось в файлах (построчно)
# Красные строки — удалено, зелёные — добавлено

git diff --staged
# То же самое, но для файлов уже добавленных через git add
```

`git status` — твой лучший кореш. Запускай постоянно. Как посмотретья в зеркало перед выходом — лишним не будет.

Добавление файлов в индекс (Staging)

Зачем вообще нужен этот промежуточный шаг?

Новичков часто путает: почему нельзя сразу `git commit` без `git add`? Что за лишний шаг?

Git разделяет работу на три зоны:

```
Рабочая папка → Индекс (Staging) → Репозиторий
(твои файлы)   (выбранные файлы)   (история)
      git add                        git commit
```

Смысл вот в чём: хороший коммит описывает **одно логическое изменение**. Представь — за вечер ты исправил баг, добавил новую кнопку и обновил README. Три разных вещи — три разных коммита. Индекс позволяет выбрать нужные файлы и закоммитить их по отдельности, не трогая остальное.

Без этого шага пришлось бы коммитить всё сразу или как-то хитрить. С индексом — просто добавляешь нужное и коммитишь.

```
git add .           # Добавить ВСЕ изменённые файлы
git add файл.txt    # Добавить конкретный файл
git add папка/      # Добавить всё из папки
git add -p          # Интерактивно — выбираешь какие куски добавить
```

▪ `git add -p` — про-мув. Изменил 5 вещей в одном файле, хочешь разбить на 2 коммита? `git add -p` позволяет выбирать куски. Сразу выглядишь как сеньор, не новичок который закоммитил всё подряд.

Делаем коммит

```
git commit -m "feat: добавил форму логина"
# -m это флаг "message" — сообщение коммита

git commit --amend
# Исправить последний коммит (сообщение или добавить файл)
# ВАЖНО: только если ещё не запустил!
```

Как писать нормальные сообщения коммитов

```
# Хорошо:
git commit -m "feat: добавил авторизацию через Google"
git commit -m "fix: исправил падение при пустом email"
git commit -m "refactor: вынес логику в отдельный сервис"

# Плохо:
git commit -m "fix"
git commit -m "aaa"
```

```
git commit -m "изменения"
git commit -m "asdfgh"
```

Хорошее сообщение коммита — это ответ на вопрос "Что делает этот коммит?". Через год ты откроешь историю и скажешь себе спасибо.

История коммитов

```
git log --oneline
# Коротко: хеш + сообщение
# Выглядит как: a1b2c3d feat: добавил авторизацию

git log --oneline --graph --all
# То же самое но с красивым деревом веток в терминале
# Обязательно попробуй — выглядит круто

git log -p
# Полная история с изменениями в каждом коммите
# Много букв, но полезно для изучения
```

Получаем изменения — fetch vs pull

Коллега запустил изменения на GitHub. Тебе надо их забрать. Есть два способа, и разница важная.

```
git fetch origin
# Скачивает все новые коммиты с сервера в локальный кэш
# Твои файлы НЕ меняются — ты просто узнаёшь что появилось

git log --oneline origin/main
# Смотришь коммиты которые есть на сервере но ещё не у тебя

git merge origin/main
# Применяешь их к своей ветке когда готов
```

```
git pull
# Это git fetch + git merge одной командой
# Скачал и сразу применил
```

`git pull` = `git fetch` + `git merge`. Это буквально две команды склеенные в одну.

Когда что юзать: - `git pull` — в большинстве случаев, стандартный способ начать день - `git fetch` — когда хочешь сначала посмотреть что прилетело прежде чем применять

Новичок делает `git pull` вслепую и удивляется конфликтам. Привычка: перед `git pull` сделай `git status` — убедись что нет незакоммиченных изменений.

Сташ — рюкзак для незаконченного кода

```
git stash
# Прячешь все незакоммиченные изменения
# Рабочая директория становится чистой

git stash push -m "описание что там лежит"
# То же самое но с именем — очень удобно когда сташей несколько

git stash pop
# Достаешь последний обратно

git stash list
# Смотришь что лежит в рюкзаке
# Покажет: stash@{0}: On main: описание что там лежит

git stash pop stash@{1}
# Достать конкретный сташ по номеру из списка

git stash drop
# Выбрасываешь последний сташ нафиг
```

«Сценарий из жизни: пишешь фичу, в Telegram прилетает "СРОЧНО ПРОД УПАЛ". Сташнул (`git stash`), пофиксил, поп — (`git stash pop`) и снова в фиче. Никто ничего не заметил.

Отмена действий — осторожно, тут можно напороться

```
git restore файл.txt
# Отменить изменения в файле (до последнего коммита)
# НЕОБРАТИМО — изменения пропадут навсегда

git restore --staged файл.txt
# Убрать файл из индекса (из "зала ожидания")
# Изменения в файле остаются, просто выходят из-под git add

git reset --soft HEAD~1
# Отменить последний коммит, изменения остаются в индексе
# ТОЛЬКО если ещё не запустил в репу!

git reset --hard HEAD~1
# Отменить последний коммит И выбросить все изменения
# НЕОБРАТИМО. Юзай только если точно знаешь что делаешь
# ТОЛЬКО если ещё не запустил в репу!
```

`git reset` на уже запущенных коммитах — это как стереть страницу из книги которую уже читает команда. Все видят что страницы нет, но не понимают почему. Не делай так.

Отменить уже запущенный коммит — `git revert`

`git reset` ломает историю когда коммит уже запущен. Для этого случая есть `git revert` — он не удаляет коммит, а создаёт новый который его отменяет. История остаётся честной, коллеги не страдают.

```
git revert HEAD
# Отменить последний коммит — создаст новый коммит-отмену
# Git откроет редактор для сообщения, можно принять по умолчанию

git revert abc1234
# Отменить конкретный коммит по хешу (хеш берёшь из git log --oneline)
```

Правило простое: не запустил — можно `reset`. Уже запустил — только `revert`.

`.gitignore` — что НЕ нужно трекать

Создай файл `.gitignore` в корне проекта:

```
# Зависимости (их можно поставить заново через npm install / pip install)
node_modules/
venv/
__pycache__/

# Логи
*.log

# Секреты — НИКОГДА не коммить это!
.env
.env.local
*.key
secrets.json

# Системный мусор
.DS_Store
Thumbs.db

# Папки IDE
.idea/
.vscode/settings.json
```

Самое страшное что можно сделать — закоммитить `.env` с паролями и токенами в публичную репу. GitHub боты сканируют репы и воруют ключи за МИНУТЫ. Спроси того парня из Твиттера который слил AWS ключи — пришёл счёт на \$50,000 за ночь.

Важно: `.gitignore` работает только для файлов которые Git ещё не трекает. Если файл уже был закоммичен раньше — просто добавить его в `.gitignore` не поможет, Git продолжит его видеть.

Нужно сначала убрать его из трекинга:

```
git rm --cached .env
# Убирает файл из Git, но оставляет его на диске
# После этого добавь .env в .gitignore и закоммить
git add .gitignore
git commit -m "chore: untrack .env and add to gitignore"
```

Задание к уроку

- В `git-practice` создай несколько файлов (например, `index.html`, `style.css`, `script.js`)
- Сделай 3–4 коммита с нормальными сообщениями в стиле Conventional Commits
- Попрактикуйся с `git status` и `git diff` — смотри что происходит на каждом шагу
- Добавь правильный `.gitignore`
- Попробуй `git stash` и `git stash pop`

Домашнее задание

- Изучи и используй `git commit --amend` — намеренно напиши опечатку в коммите и исправь её
- Попробуй `git add -p` — измени несколько вещей в одном файле и добавь только часть

Типичные ошибки новичка

Ошибка	Что происходит
<code>git reset --hard</code> на важных изменениях	Код пропал навсегда, плачешь
<code>git reset</code> на запущенных коммитах	Ломаешь историю для всей команды
Закоммитил <code>.env</code> с паролями	Меняешь все пароли срочно
<code>git add .</code> не глядя	Коммитишь мусор, логи, скомпилированные файлы
Пишешь <code>fix</code> в каждом коммите	Через месяц не понимаешь что вообще происходило

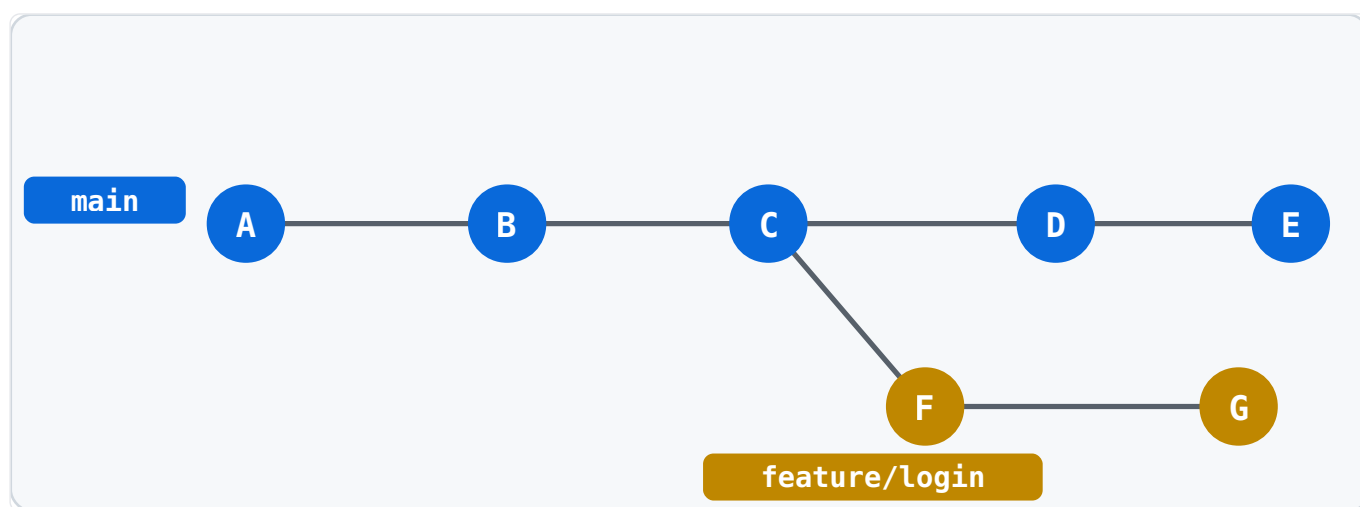
Урок 5: Ветки — работаешь без страха

- **TL;DR:** `git switch -c feature/моя-фича` → пишешь код → коммитишь → `git switch main` → `git merge feature/моя-фича`. Готово, не сломал продакшн.

Что такое ветки и зачем они нужны?

Представь: у тебя есть работающий сайт в `main`. Тебе нужно добавить новую фичу. Если писать прямо в `main` — и фича сырая, и прод сломан. Все страдают.

Ветки решают это. Ты создаёшь параллельную вселенную (`feature/login`), пишешь там что угодно, и только когда всё готово и протестировано — сливаешь обратно в `main`.



Синие кружки — коммиты в `main`. Оранжевые — в `feature/login`. Обе ветки живут параллельно и не мешают друг другу.

*Ветки бесплатные и быстрые. Создать бранч — одна команда и доля секунды. Бранчись на каждую задачу. Это не оверхед — это правильный стиль.

Основные команды для веток

Раньше для всего юзали `git checkout`. Это старый способ, он ещё работает, но с Git 2.23 (2019) появились специальные команды `git switch` и `git restore`. Учи новый синтаксис — он понятнее.

```
git branch
# Показать все локальные ветки (* — где ты сейчас)

git branch -a
# Показать все ветки включая удалённые

git switch main
# Переключиться на ветку main
```

```
git switch -c feature/login
# Создать новую ветку И сразу переключиться на неё
# -c = create (старый аналог: git checkout -b feature/login)

git branch -d feature/old
# Удалить ветку (только если уже с мержена)

git branch -D feature/old
# Удалить ветку принудительно (даже если не с мержена)
# Данные потеряются если не запустил
```

Основной workflow с ветками

Вот как выглядит нормальная работа над задачей:

```
# 1. Убеждаемся что main актуальный
git switch main
git pull
# Тут важно: всегда делай pull перед созданием ветки
# Иначе твоя ветка будет отставать от реальности

# 2. Создаём ветку с понятным именем
git switch -c feature/user-registration
# Соглашение по именам:
# feature/название — новая фича
# fix/название — исправление бага
# hotfix/название — срочный фикс в проде
# refactor/название — рефакторинг

# 3. Работаем, коммитим как обычно
git add .
git commit -m "feat: добавил форму регистрации"
git commit -m "feat: добавил валидацию email"
git commit -m "fix: исправил падение при пустом пароле"

# 4. Возвращаемся на main
git switch main

# 5. Сливаем нашу ветку
git merge feature/user-registration
# Git скажет что с мержил — история сохраняется

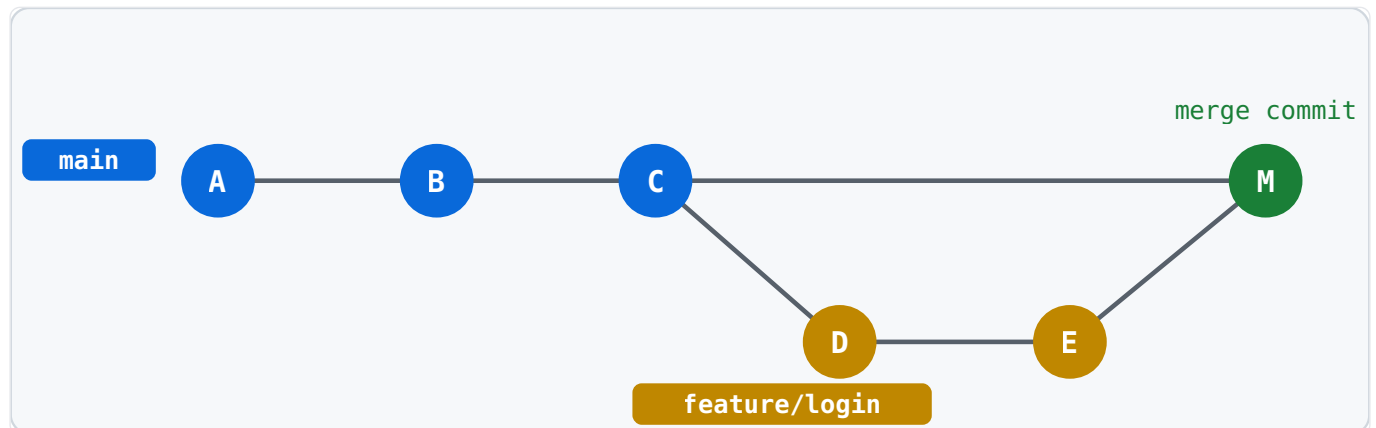
# 6. Чистим за собой
git branch -d feature/user-registration
```

Называй бранчи понятно. *feature/add-google-auth* лучше чем *feature/auth* и в 100 раз лучше чем *test2* или *ветка-дениса*. Все видят название в репе — уважай людей.

Merge vs Rebase — два способа слить

Merge — простой и честный

```
git switch main
git merge feature/login
# Создаёт "merge commit" — запись что две ветки слились
# История выглядит как настоящее дерево с развилками
```



Коммит M (зелёный) — это `merge commit`. У него два родителя: C из `main` и E из `feature`. История честно показывает что была отдельная ветка.

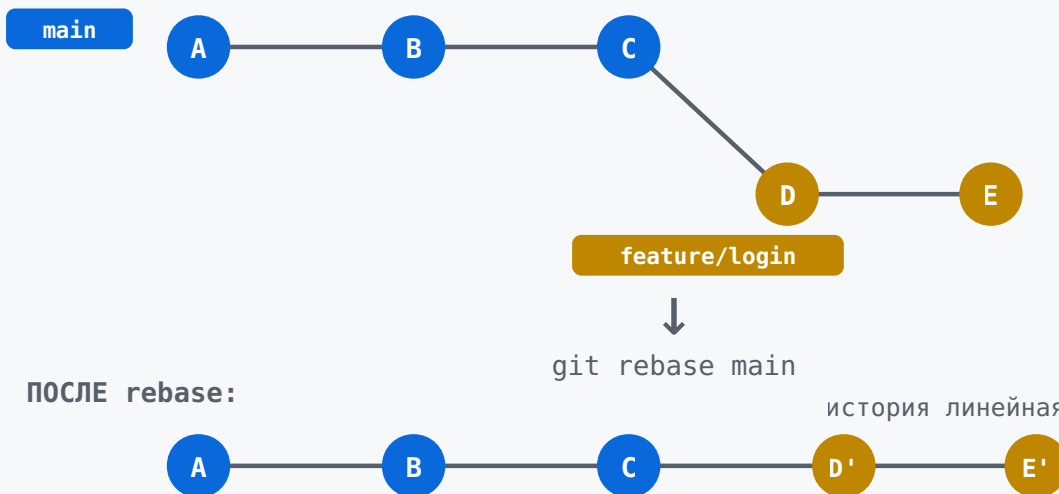
Merge с `--no-ff` — сохраняет структуру

```
git merge --no-ff feature/login
# Всегда создаёт merge commit даже если можно было без него
# Видно что была отдельная ветка — удобно смотреть историю
```

Rebase — чистый и красивый

```
git switch feature/login
git rebase main
# "Переставляет" твои коммиты поверх свежего main
# История выглядит линейно — как будто всё шло последовательно
```

ДО rebase:



D и E стали D' и E' — содержимое то же, но они «переставлены» поверх C. История линейная, как будто ветки не было.

Золотое правило rebase: *ребейзь только те ветки, которые видишь только ты. Если ветка уже запущена и кто-то другой её скачал — `git rebase` перепишет историю, и у коллеги сломается реп.*
Безопасная схема: ребейзь локально до первого `push`, после — только `merge`.

**Один на проекте — юзай что хочешь. В команде договариваются заранее. Классика: ребейзишь локально, мержишь финально через ПР.*

Merge Conflicts — не паникуй, это нормально

Конфликт случается когда два человека (или две ветки) изменили один и тот же кусок кода. Git не знает какую версию оставить — просит тебя решить.

```
git merge feature/login
# Auto-merging index.html
# CONFLICT (content): Merge conflict in index.html
# Automatic merge failed; fix conflicts and then commit the result.
```

Открой файл с конфликтом — увидишь такое:

```
<<<<<<< HEAD
<h1>Добро пожаловать на сайт</h1>
=====
<h1>Добро пожаловать, пользователь!</h1>
>>>>>> feature/login
```

- `<<<<<<< HEAD` до `=====` — это твоя версия (текущая ветка)
- `=====` до `>>>>>>>` — это версия из мерджимой ветки

Ты выбираешь что оставить (или комбинируешь), убираешь маркеры `<<<`, `===`, `>>>`, сохраняешь:

```
git add index.html
git commit
# Git предложит сообщение "Merge branch 'feature/login'" — можешь принять
```

VS Code и другие редакторы подсвечивают конфликты и дают кнопки "Accept Current / Accept Incoming / Accept Both". Юзай их — удобнее чем вручную.

■

■

■ Задание к уроку

1. ■ Создай ветку `feature/about` командой `git switch -c feature/about`
2. ■ Добавь файл `about.html` с каким-нибудь содержимым
3. ■ Сделай 2-3 коммита
4. ■ Слей ветку в `main` через `git merge --no-ff feature/about`
5. ■ Намеренно создай конфликт: измени одну строку в `main` и в фича-ветке, попробуй слить

■ Домашнее задание

- Попробуй `git rebase` вместо `merge` — посмотри как выглядит история через `git log --oneline --graph --all`
- Сравни результат: с `merge` получается дерево, с `rebase` — прямая линия

■ Типичные ошибки новичка

Ошибка	Что происходит
Пишешь код прямо в <code>main</code>	Сломал прод, тимлид недоволен
Создаёшь ветку от старого <code>main</code>	Мержишь в актуальный — тонна конфликтов
Называешь ветку <code>test</code> или <code>ветка1</code>	Никто не понимает что там происходит
Не удаляешь ветки после мержа	Репозиторий обрастает мусором из 50 веток
<code>git rebase</code> на общей ветке которую видят другие	Переписываешь историю, у коллег сломается репозиторий

Урок 6: GitHub и командная работа

🔔 **TL;DR:** Создал репу на GitHub → связал с локальной → `git push` → открыл PR → получил ревью → замержил. Добро пожаловать в командную разработку.

GitHub — зачем он вообще нужен?

Без GitHub твой код живёт только на твоём компьютере. Упадёт жёсткий диск — пока код. Хочешь показать работодателю — не можешь. Хочешь работать в команде — пишете на флешках?

GitHub решает всё это: - Бекап кода в облаке - Совместная работа с командой - Код-ревью через Pull Requests - Трекинг задач и багов через Issues - Твоё публичное портфолио для работодателей

🔔 GitHub — LinkedIn для разработчиков. Только вместо "работал в Газпроме 5 лет" там "вот мои репы, смотри сам что умею".

Создание репозитория на GitHub

1. Зайди на github.com → войди в аккаунт
2. Нажми + → **New repository**
3. Заполни:
4. **Repository name:** `git-practice` (такое же как локальная папка)
5. **Description:** можно написать что-нибудь
6. **Public / Private:** Public если хочешь показывать людям
7. **НЕ** ставь галочки "Add README", "Add .gitignore" — у тебя уже есть локальный репозиторий с файлами
8. Нажми **Create repository**

🔔 Если поставишь README при создании — GitHub сделает первый коммит, и при попытке запустить свой код получишь конфликт истории. Неприятно для новичка.

Связываем локальный репозиторий с GitHub

GitHub покажет инструкцию после создания репы. Вот что нужно сделать:

```
cd git-practice

# Говоришь Git где живёт удалённая репа
# "origin" — стандартное имя, можно назвать как угодно но все называют origin
git remote add origin https://github.com/ТВО_ЛОГИН/git-practice.git
# Или через SSH (если настроил ключи):
git remote add origin git@github.com:ТВО_ЛОГИН/git-practice.git
```

```
# Переименовываем ветку в main (если ещё не сделал)
git branch -M main

# Первый пуш с флагом -u — устанавливает "upstream"
# После этого можно просто писать git push без параметров
git push -u origin main
```

Проверь что всё связалось:

```
`git remote -v
# Должно показать origin с твоей ссылкой (fetch и push)
```

Команды для работы с удалённым репозиторием

```
git push
# Отправить свои коммиты на GitHub
# После первого git push -u origin main можно просто git push

git pull
# Скачать изменения с GitHub и сразу смержить в текущую ветку
# = git fetch + git merge
# Делай это каждое утро перед началом работы!

git fetch
# Скачать информацию об изменениях НО не мержить
# Полезно чтобы посмотреть что изменилось у коллег, не трогая свой код
# git fetch → git log origin/main → понял что там → git merge или не мержишь

git clone https://github.com/логин/репо.git
# Скачать чужую (или свою) репу с нуля
# Создаёт папку с именем репы, уже настроенную с origin
```

fetch vs pull — это как "посмотреть меню" vs "заказать блюдо". fetch просто смотрит что есть на сервере. pull — скачивает и сразу смешивает с твоим кодом. Если не уверен что у коллег там — лучше сначала fetch, потом log, потом решаешь.

Pull Request — сердце командной работы

Pull Request (PR) — это способ сказать команде "Эй, я написал фичу, посмотрите, если норм — смержим в main".

Создаём PR

```
# 1. Работаем в ветке
git switch -c feature/cool-feature
```

```
# 2. Делаем коммиты
git add .
git commit -m "feat: добавил крутую фичу"

# 3. Пушим ветку на GitHub
git push origin feature/cool-feature
# Или если уже делал -u:
git push
```

Потом на GitHub: 1. Появится жёлтый баннер **"Compare & pull request"** — жми его 2. Напиши что сделал — **заголовок** (кратко) и **описание** (что изменил, почему, как тестировал) 3. Назначь ревьюера если есть (кнопка **Reviewers** справа) 4. Нажми **Create pull request**

После ревью

- Ревьюер оставляет комментарии → ты правишь код → делаешь новые коммиты в ту же ветку → PR обновляется автоматически
- Когда все довольны → кнопка **Merge pull request** → ветка сливается в `main`
- Удали ветку (GitHub предложит кнопку после мержа)

Хороший ПР — это не просто код. Это объяснение ЗАЧЕМ этот код. "Добавил кнопку" — плохое описание. "Добавил кнопку экспорта в CSV по запросу клиентов, работает через новый endpoint /api/export" — норм. Ревьюер скажет спасибо и замерзит быстрее.

Fork — работаем с чужими проектами

Fork нужен когда хочешь поконтрибьютить в чужой проект (open source) или взять чужой код за основу.

```
# 1. На GitHub жмёшь Fork на чужой репе — появляется твоя копия
# 2. Клонировать свою копию
git clone git@github.com:ТВО_ЛОГИН/чужой-проект.git
cd чужой-проект

# 3. Добавляешь оригинальную репу как upstream
git remote add upstream git@github.com:ОРИГИНАЛ/чужой-проект.git

# 4. Синкаешься с оригиналом когда нужно
git fetch upstream
git merge upstream/main

# 5. Создаёшь ветку, делаешь изменения, пушишь в свой форк
# 6. Открываешь PR из своего форка в оригинальную репу
```

Полезные фишки GitHub

- **Issues** — трекер задач. Создай issue на каждую задачу/баг. В описании PR пиши `fix #42` — автоматически закроет issue когда PR смержат в `main`. Важно: в теле коммита это тоже работает, но только после мержа PR в ветку по умолчанию, не сразу после пуша
- **README.md** — лицо репозитория. Пиши туда что это, как запустить, как использовать
- **GitHub Actions** — автоматические тесты и деплой при каждом пуше. Мощная штука, отдельная тема
- **Защита веток** — в Settings можно запретить пуш напрямую в `main`. Только через PR. Хорошая практика для командных проектов

Задание к уроку

1. Создай репозиторий на GitHub
2. Свяжи с локальным `git-practice`
3. Запушь все свои коммиты
4. Создай ветку `feature/readme-update`, улучши README, запушь и открой PR
5. Сам же замержи PR (в реальности это делает другой человек, но для практики сойдёт)

Финальное домашнее задание

- Создай **публичную репу** для какого-нибудь своего небольшого проекта (даже если это просто "Hello World")
- Напиши нормальный `README.md`: что это, как запустить, скриншот если есть
- Пришли ссылку учителю — это уже твоё первое публичное портфолио

Типичные ошибки новичка

Ошибка	Что происходит
<code>git push</code> в форкнутую репу, а не в свою	Ошибка доступа или пушишь не туда
PR с 40 коммитами "fix", "fix2", "fix fix"	Ревьюер плачет, мерж откладывается
Не синкаешь форк с апстримом	Твоя копия отстаёт на месяц, тонна конфликтов
<code>git push --force</code> в общую ветку	Удаляешь чужие коммиты. Тебя не любят в команде
Не читаешь комментарии к PR	Ревьюер написал важное, ты смержил не глядя

Ты прошёл весь курс!

Теперь ты знаешь Git достаточно чтобы работать в команде и не быть тем человеком который коммитит прямо в `main`.

Дальше — практика, практика и ещё раз практика. Форкни какой-нибудь open source проект и отправь PR. Даже если это просто фикс опечатки в документации — это уже реальный контриб!

Урок 7: Git в VSCode — всё то же самое, но кликами

***TL;DR:** Всё что ты делал в терминале — `add`, `commit`, `diff`, ветки, конфликты — VSCode умеет делать мышкой. Это не замена терминалу, это второй способ делать то же самое.*

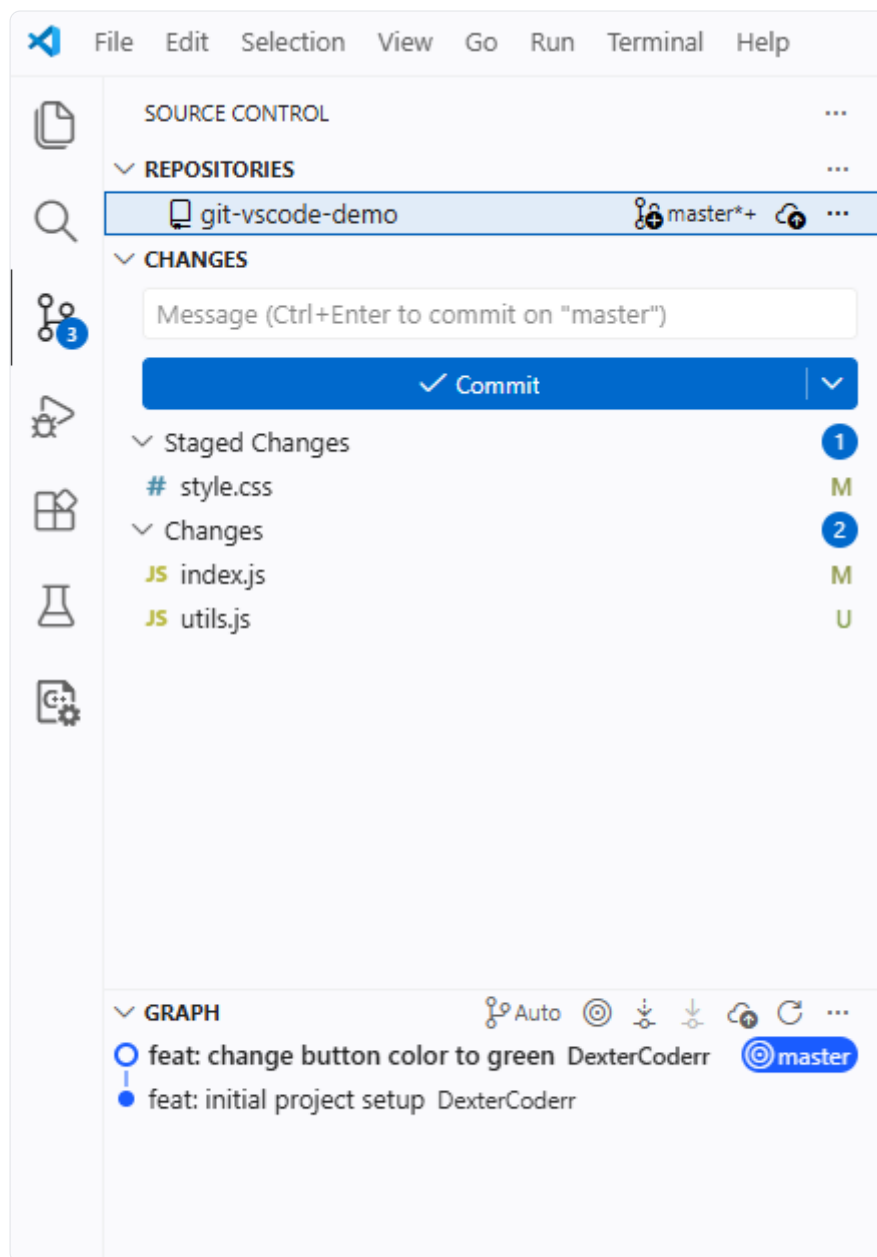
Ты уже знаешь команды. Ты уже коммитил через терминал и не умер. Теперь секрет который не говорят вслух:

Большинство разработчиков делают рутинный Git мышкой прямо в редакторе. А терминал открывают только для сложных вещей — `rebase`, `cherry-pick`, разруливание серьёзных конфликтов.

VSCode умеет в Git из коробки — без плагинов, без настройки. Просто открываешь папку с репозиторием — и всё работает.

Где это всё живёт — Source Control панель

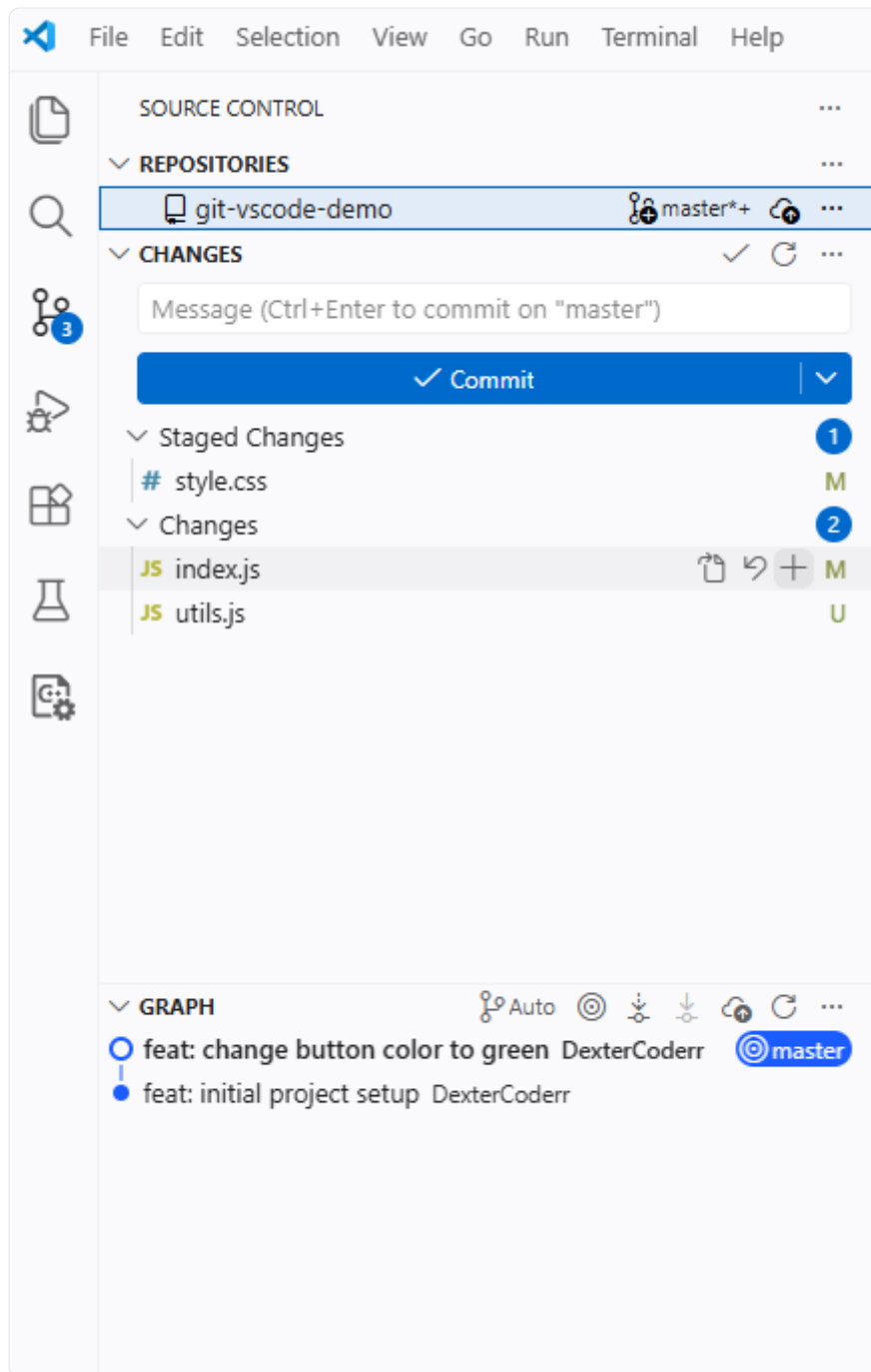
Нажми **Ctrl+Shift+G** (на macOS — **Cmd+Shift+G**), или кликни на иконку веток в левом сайдбаре:



Что видишь: - **Staged Changes** — файлы добавленные в индекс, готовые к коммиту (`git add` уже сделан) - **Changes** — файлы изменённые, но ещё не добавленные в индекс - **M** — Modified, изменён; **U** — Untracked, Git не знает про этот файл - **Commit** — кнопка закоммитить всё что в Staged Changes - Стрелка ▼ рядом с Commit — выпадает меню с вариантами (Commit & Push, Commit & Sync)

Stage — добавляем файлы в индекс

В терминале ты делал `git add файл`. В VSCode — наводишь на файл и жмёшь +:



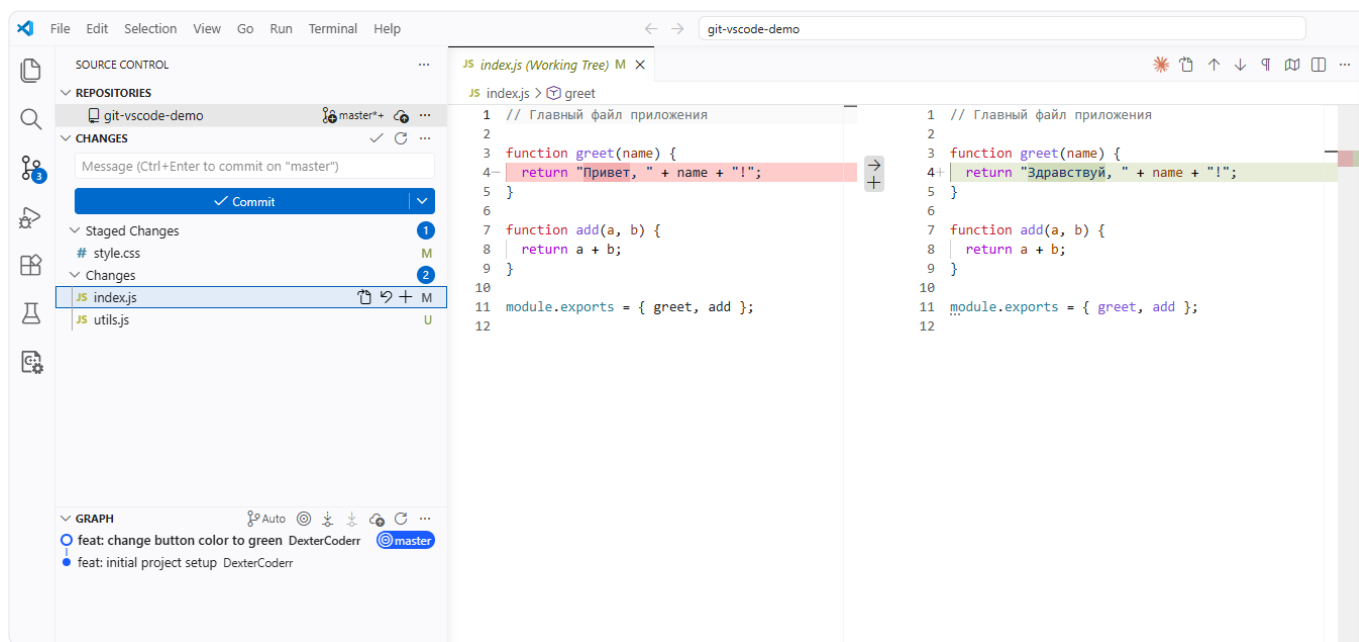
Видишь иконки которые появились справа на `index.js` : - — открыть файл - `↶` — отменить изменения (`git restore`) - `+` — добавить в индекс (`git add`)

После клика на `+` файл переедет из **Changes** в **Staged Changes**.

Ты можешь добавить только `index.js`, а `utils.js` оставить на потом. Именно за это и нужен двухшаговый процесс — один коммит про одно изменение, не всё подряд.

Diff — смотрим что изменилось

Кликни на любой файл в списке Changes — откроется diff прямо в редакторе:

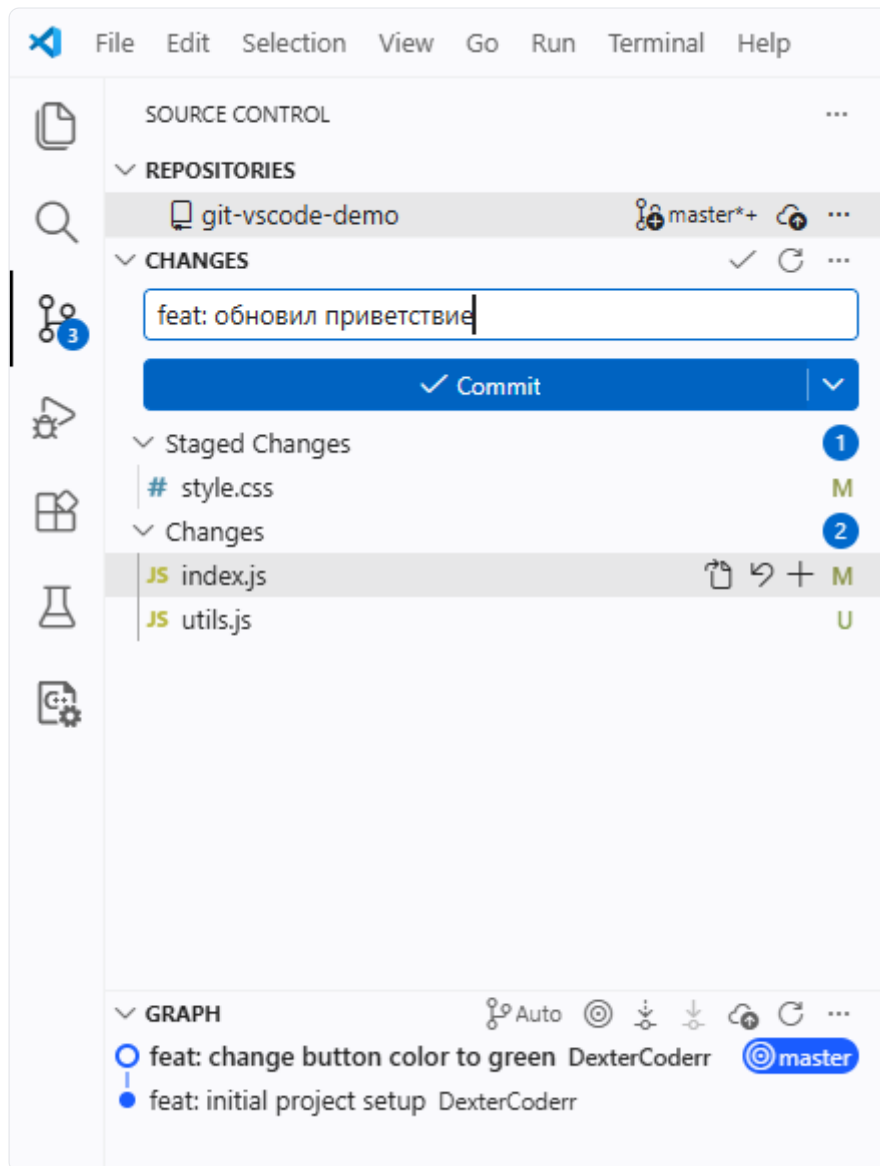


Слева — было, справа — стало. Красный фон — удалено, зелёный — добавлено. То же самое что `git diff` в терминале, только сразу глазами.

“Смотри diff перед каждым коммитом. 10 секунд — и не закоммитишь случайный `console.log` в продакшн.

Commit — коммитим через интерфейс

Напиши сообщение коммита в поле сверху и нажми **Commit**:

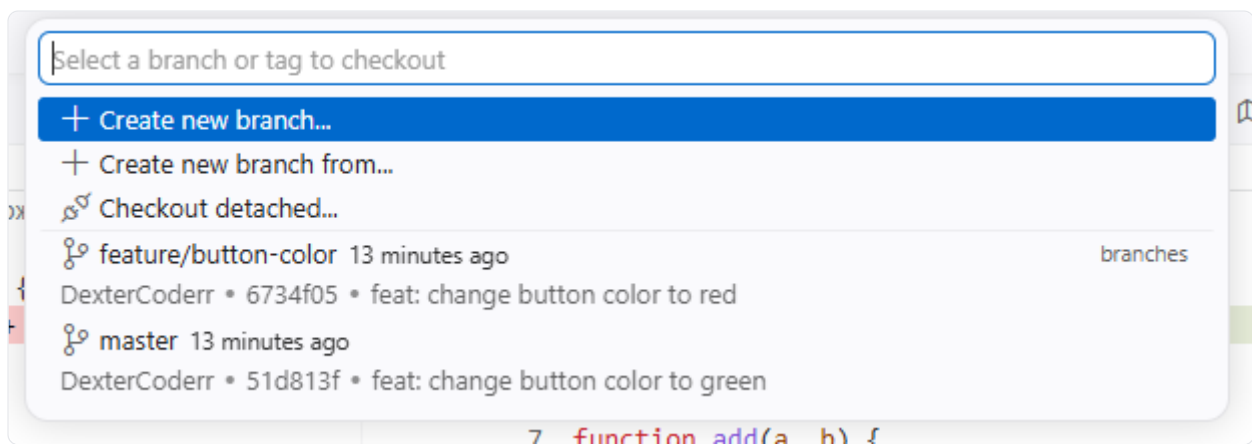


Стрелка ▼ рядом с кнопкой открывает варианты: - **Commit** — просто закоммитить - **Commit & Push** — закоммитить и сразу запустить - **Commit & Sync** — закоммитить, подтянуть чужие изменения, запустить

🌱 "Commit & Sync" удобен в командной работе — делает *pull* + *push* одним кликом. Но если есть конфликты, придётся разруливать.

Ветки — переключаемся без терминала

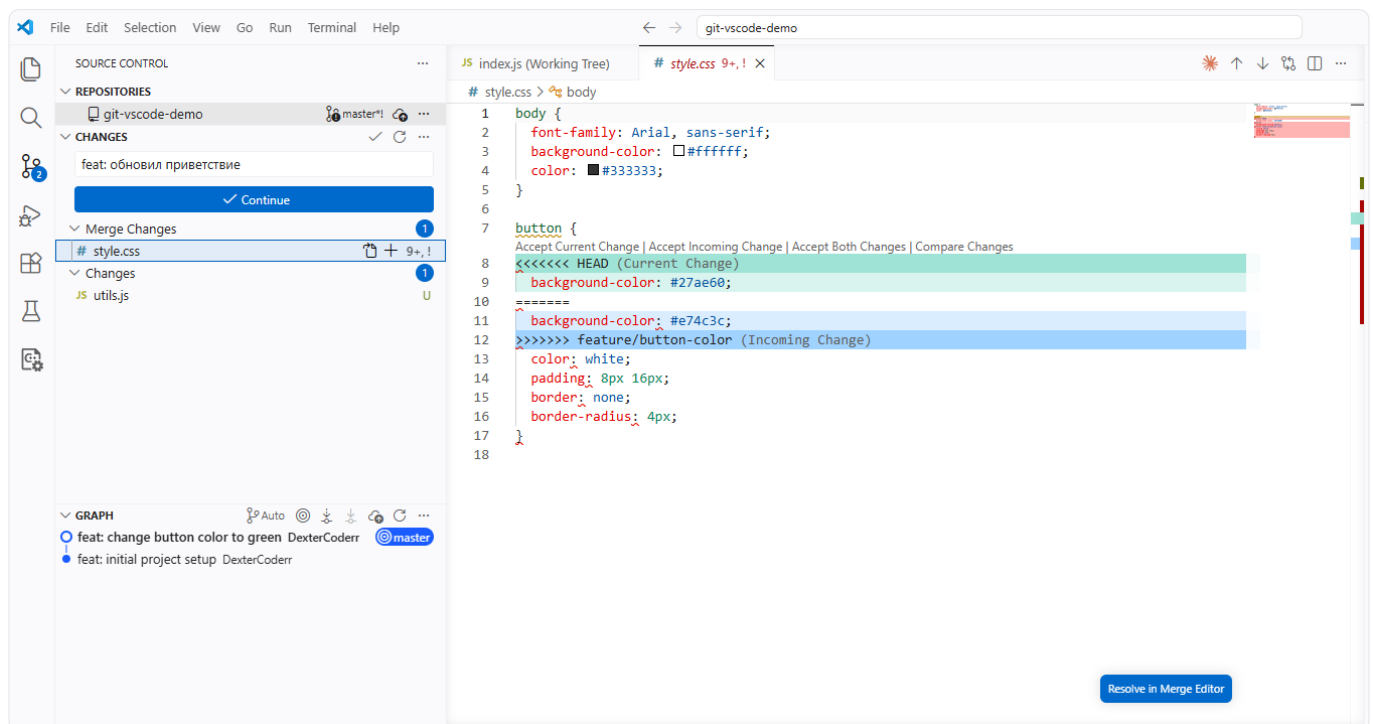
Кликни на название текущей ветки рядом с именем репозитория — откроется список:



Видишь: - **Create new branch...** — создать новую ветку - Список существующих веток с последним коммитом и хешем - Кликаешь на ветку — переключаешься мгновенно (`git switch`)

Конфликты — это не страшно когда видишь их глазами

В терминале конфликт выглядит как три куска непонятного текста с тегами `<<<` и `>>>`. В VSCode та же ситуация выглядит по-человечески:



Вверху конфликтного блока появляются ссылки: - **Accept Current Change** — оставить твоё изменение (зелёный блок, HEAD) - **Accept Incoming Change** — взять чужое изменение (синий блок) - **Accept Both Changes** — оставить оба варианта - **Compare Changes** — посмотреть diff

Кнопка **Resolve in Merge Editor** внизу справа — открывает трёхпанельный редактор для сложных конфликтов.

Большинство конфликтов решаются за 30 секунд когда видишь их вот так. В терминале та же операция занимает 5 минут нервных поисков тегов `<<<`.

Задание к уроку

1. Открой свой `git-practice` в VSCode
2. Измени пару файлов и найди их в Source Control панели
3. Добавь один файл через `+` и посмотри как он переехал в Staged Changes
4. Сделай коммит через интерфейс — убедись что сообщение нормальное
5. Кликни на изменённый файл и изучи diff — найди что именно изменилось

Типичные ошибки

Ошибка	Что происходит
Нажал "Commit" без сообщения	VSCode ругается, коммита нет — хорошо что не пропустил
Нажал "Commit & Sync" с конфликтами	VSCode остановится и покажет конфликты, не страшно
Не смотришь diff перед коммитом	Коммитишь лишнее: дебаг-код, console.log, случайные изменения
Думаешь что VSCode-кнопки — это что-то другое	Это буквально те же git-команды, просто без терминала

Важно: VSCode не заменяет знание команд — он их прячет за кнопками. Поэтому ты сначала учил терминал. Теперь ты понимаешь что происходит за каждым кликом — это и есть разница между "нажал кнопку" и "знаю что делаю".